2

Лекция 16–17 февраля

Обходы и деревья

Константин Чухарев

Дискретная математика — Весна 2027

Обходы графа

> *Не все, кто блуждает, потеряны.*
>
> ***— Дж. Р. Р. Толкин***

Зачем обходить граф?

Граф задан, но из пары множеств мало что видно.

Обход превращает граф в структуру с порядком посещения.

- Найти компоненты связности.
- Найти кратчайшие пути от старта.
- Найти циклы и топологический порядок.
- Построить остов.

Обход BFS или DFS стоит $O(V + E)$. Каждая вершина посещается один раз, а каждое ребро просматривается с обоих концов.

Всюду ниже граф хранится списками смежности. Сумма длин всех списков равна $2\,|E|$, поэтому линейность достижима.

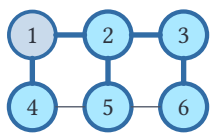
На матрице смежности один только просмотр соседей стоит уже $O(V)$ на вершину.

Поиск в ширину

BFS расходится от старта волной, а очередь хранит вершины текущего слоя.

1. Помещаем стартовую вершину в очередь.
2. Пока очередь не пуста, повторяем два шага.
   - Извлекаем вершину u из головы очереди.
   - Всех непосещённых соседей v помечаем и ставим в хвост.

Вершины выходят из очереди в порядке поступления, поэтому ближние слои обрабатываются раньше дальних.



BFS идёт слоями. Сначала в волну попадают соседи старта, затем соседи соседей, а внутри слоя порядок произвольный.

Трассировка BFS по шагам

Пройдём алгоритм по решётке BFS. Старт — вершина 1, соседи перебираются по возрастанию номеров.

Шаг	Извлечена	Обнаружены	Очередь после шага
1	—	1	[1]
2	1	2, 4	[2, 4]
3	2	3, 5	[4, 3, 5]
4	4	—	[3, 5]
5	3	6	[5, 6]
6	5	—	[6]
7	6	—	[]

Очередь хранит вершины не более двух соседних слоёв, и слои идут подряд.

Дерево обхода и расстояния

Пример (Итоги прогона)

Вершины извлекаются в порядке $1, 2, 4, 3, 5, 6$.

Расстояние от старта равно $d(1) = 0$, $d(2) = d(4) = 1$, $d(3) = d(5) = 2$ и $d(6) = 3$.

Родители вершин заданы равенствами $\mathrm{parent}(2) = \mathrm{parent}(4) = 1$, а дальше $\mathrm{parent}(3) = \mathrm{parent}(5) = 2$ и $\mathrm{parent}(6) = 3$.

Путь до 6 собирается с конца по родительским ссылкам и даёт $1 \to 2 \to 3 \to 6$.

Родительские ссылки образуют дерево кратчайших путей, по ним восстанавливается маршрут до любой вершины.

Вершина попадает в очередь ровно один раз, при первом обнаружении, поэтому в колонке обнаружений каждая вершина встречается один раз.

Почему BFS идёт кратчайшими путями?

Пока $d(v)$ — лишь номер слоя, а теорема отождествляет его с расстоянием.

Теорема 1 (Корректность расстояний BFS)

В невзвешенном графе $d(v)$ равно длине кратчайшего пути из s в v для каждой достижимой вершины v.

Доказательство

Значение $d(v)$ — длина настоящего пути, поэтому $d(v) \geq \mathrm{dist}(v)$. Обратное неравенство докажем индукцией по слою, попутно проверяя порядок очереди.

База. Слой 0 — один старт, и он выходит первым.

Шаг. Пусть $\mathrm{dist}(v) = k + 1$, а u — предпоследняя вершина кратчайшего пути до v, тогда $d(u) = k$. К обработке u вершина v либо бела и получает $d(v) = k + 1$, либо обнаружена раньше, и тогда $d(v) \leq k + 1$.

Слои и кратчайшие пути

В социальной сети слои BFS образуют круги знакомых. В первый круг входят друзья, во второй — друзья друзей, в третий — друзья друзей друзей.

Пример (Шесть рукопожатий)

В эксперименте Милгрэма (1967) незнакомцам доверяли письма, и каждый участник передавал конверт знакомому, который казался ближе к адресату.

Цепочки дошедших конвертов содержали в среднем около шести передач — слои BFS измеряют те же «рукопожатия» в социальной сети.

Пусть ребро $s - a$ имеет вес 10, а рёбра $s - b$ и $b - a$ — вес 1. BFS обнаружит a прямо из s и вернёт путь веса 10, хотя путь $s - b - a$ весит 2.

Взвешенный минимум ищет Дейкстра.

Компоненты за один обход

Компоненты связности считаются обходом.

Запуск из непосещённой вершины собирает ровно её компоненту.

Повторяем запуски, пока остаются непосещённые вершины.

Пример (Две компоненты)

Пусть в графе есть рёбра $1 - 2$, $2 - 3$, $3 - 4$ и отдельно $5 - 6$, $6 - 7$.

Запуск BFS из 1 даёт порядок $1, 2, 3, 4$ — компонента собрана целиком.

Вершина 5 осталась белой, поэтому второй запуск даёт $5, 6, 7$ — итого две компоненты.

Число запусков равно числу компонент, а общее время остаётся $O(V + E)$. Каждая вершина по-прежнему посещена один раз.

Двудольность и раскраска из BFS

BFS красит слои попеременно. Старт получает цвет A, его соседи получают цвет B, а следующий слой снова возвращается к A.

Пример (Раскраска решётки)

Трассировка даёт слои $\{1\}$, затем $\{2, 4\}$, затем $\{3, 5\}$, затем $\{6\}$.

Чётные слои дают долю $A = \{1, 3, 5\}$, нечётные — долю $B = \{2, 4, 6\}$.

Все семь рёбер соединяют разные доли — разбиение получено одним обходом.

Раскраска по чётности слоя корректна ровно тогда, когда внутри слоёв нет рёбер. Для трёх цветов такого критерия нет.

Критерий двудольности

Нечётный цикл оказывается единственной помехой двум цветам.

Теорема 2 (Критерий двудольности)

Граф двудолен тогда и только тогда, когда в нём нет циклов нечётной длины.

Доказательство

Двудольность запрещает нечётные циклы. Доли вдоль цикла чередуются, и замыкающее ребро $v_k - v_1$ соединяет разные доли, поэтому k чётен.

Отсутствие нечётных циклов даёт раскраску. Покрасим каждую компоненту по чётности слоя BFS от её старта. Ребро $x - y$ одного цвета дало бы замкнутый обход длины $d(x) + d(y) + 1$, нечётной при равной чётности слагаемых. Самый короткий такой обход — цикл, иначе повтор вершины разрезал бы его на более короткий нечётный кусок. Нечётных циклов нет, поэтому рёбер внутри слоя не бывает.

Пятиугольник как нечётный цикл

Пример (Пятиугольник)

Пусть в графе есть рёбра $1-2$, $2-3$, $3-4$, $4-5$ и $5-1$.

BFS из 1 даёт слои $\{1\}$, затем $\{2,5\}$, затем $\{3,4\}$.

По чётности слоя получаются доли $A=\{1,3,4\}$ и $B=\{2,5\}$, а ребро $3-4$ лежит внутри A.

Цикл $1-2-3-4-5-1$ имеет длину 5, и два цвета ему не подходят ни в каком назначении.

Конфликт может обнаружиться не в первом слое. Начало раскраски выглядит безупречно, а противоречие ждёт на дальнем слое.

Вывод о двудольности законен только после полной раскраски всей компоненты.

Поиск в глубину

DFS уходит вглубь по первому непосещённому соседу. Возврат случается только в тупике, когда белых соседей не осталось.

1. Для каждой непосещённой вершины u запускаем $\mathrm{DFS}(u)$.
2. Функция $\mathrm{DFS}(u)$ делает три вещи.
   - Помечает u и записывает время входа $\mathrm{pre}(u)$.
   - Для каждого непосещённого соседа v вызывает $\mathrm{DFS}(v)$.
   - Записывает время выхода $\mathrm{post}(u)$.

Структурой данных служит стек вызовов. Уходим вглубь, пока это возможно.

DFS не гарантирует кратчайших путей. Порядок обхода задаёт стек вызовов, и его глубина равна длине текущего пути.

Спуск в трассировке DFS

Стартуем из вершины 1 и перебираем соседей по возрастанию номеров. Списки смежности выглядят так. $1 : [2, 4]$, $2 : [1, 3, 5]$, $3 : [2, 6]$, $4 : [1, 5]$, $5 : [2, 4, 6]$, $6 : [3, 5]$.

Шаг	Событие	Стек вызовов	Время
1	вход в 1	1	$\mathrm{pre}(1) = 1$
2	1 зовёт 2	1, 2	$\mathrm{pre}(2) = 2$
3	2 зовёт 3	1, 2, 3	$\mathrm{pre}(3) = 3$
4	3 зовёт 6	1, 2, 3, 6	$\mathrm{pre}(6) = 4$
5	6 зовёт 5	1, 2, 3, 6, 5	$\mathrm{pre}(5) = 5$
6	5 зовёт 4	1, 2, 3, 6, 5, 4	$\mathrm{pre}(4) = 6$

Стек вызовов — это текущий путь от старта, так что колонка стека читается как маршрут $1, 2, 3, 6, 5, 4$.

Возвраты в трассировке DFS

Тупик запускает откат. Вызов закрывается, и управление получает ближайшая живая развилка.

Шаг	Событие	Стек после шага	Время
7	у 4 нет белых соседей — выход	$1, 2, 3, 6, 5$	$\mathrm{post}(4) = 7$
8	5 досмотрела соседей — выход	$1, 2, 3, 6$	$\mathrm{post}(5) = 8$
9	выход из 6	$1, 2, 3$	$\mathrm{post}(6) = 9$
10	выход из 3	$1, 2$	$\mathrm{post}(3) = 10$
11	2 видит чёрную 5 — выход	1	$\mathrm{post}(2) = 11$
12	1 видит чёрную 4 — выход	$[]$	$\mathrm{post}(1) = 12$

Обход даёт порядок $1, 2, 3, 6, 5, 4$ — глубина вместо ширины.

Рёбра $2 — 5$ и $1 — 4$ — это контакты с серыми предками при переборе соседей вершин 5 и 4.

Какие рёбра нашёл обход?

Пример (Два сорта рёбер)

Дерево образуют пять спусков из таблицы, а именно рёбра $1 — 2$, $2 — 3$, $3 — 6$, $6 — 5$ и $5 — 4$.

Два обратных ребра, $2 — 5$ и $1 — 4$, пришли из контактов с серыми предками.

Родитель вершины 5 — это 6, а серый сосед 2 лежит выше по стеку, поэтому ребро $2 — 5$ ведёт к предку.

У вершины 4 родитель 5, и серый сосед 1 снова лежит выше по стеку — ребро $1 — 4$ тоже ведёт к предку.

В неориентированном графе всякое недревесное ребро — обратное, и в каждом цикле такое ребро есть.

Белый, серый и чёрный цвета обхода

Трассировка различала вершины по судьбе в стеке — закрепим различие тремя цветами.

Определение 1 (Цвета вершин)

Вершины при обходе получают один из трёх цветов.

1. **Белый** — вершина ещё не обнаружена.
2. **Серый** — вершина обнаружена, но её вызов ещё не закрыт.
3. **Чёрный** — вершина закрыта, обход вышел из неё и не вернётся.

Серые вершины — это в точности стек вызовов, то есть путь от текущей вершины до корня. Проверка серости соседа равносильна проверке, лежит ли он на текущем пути.

В коде цвета сжимаются в булев флаг `visited`. Для BFS это безболезненно, но в ориентированном графе теряется различие между обратным ребром и ребром в закрытую вершину.

Что означает ребро в серую вершину?

Серый сосед — это возврат на текущий путь, а такой возврат замыкает цикл.

Теорема 3 (Обратные рёбра и циклы)

DFS встречает ребро к серой вершине, не считая ребра к родителю, тогда и только тогда, когда в графе есть цикл.

Доказательство

Серое ребро даёт цикл. Серые вершины — это стек вызовов, поэтому v — предок u, а путь по дереву вместе с ребром $u - v$ замыкается в цикл.

Цикл даёт серое ребро. Ребро в чёрную вершину невозможно, иначе она при своём переборе увидела бы белую u и забрала ребро в дерево. Значит, недревесное ребро ведёт к серому предку, а цикл без недревесных рёбер не обходится.

Где обход увидел циклы?

Пример (Циклы из трассировки)

Ребро $2 — 5$ замыкает цикл $2 — 3 — 6 — 5 — 2$.

Ребро $1 — 4$ замыкает цикл $1 — 2 — 3 — 6 — 5 — 4 — 1$.

Оба контура простые, ведь путь по дереву не повторяет вершин.

Связный граф без циклов — это дерево, так что проверка ацикличности и построение остова — один и тот же обход.

Для леса обходы запускаются, пока остаются белые вершины.

Обходы

1. Почему вершина помечается при входе в очередь, а не при выходе из неё?
2. Как по таблице трассировки DFS узнать обратное ребро?

Времена pre и post

Для каждой вершины интервал $[\mathrm{pre}(u), \mathrm{post}(u)]$ — время её жизни в обходе.

Теорема 4 (Вложенность интервалов)

Для любых вершин u, v интервалы $[\mathrm{pre}(u), \mathrm{post}(u)]$ и $[\mathrm{pre}(v), \mathrm{post}(v)]$ либо не пересекаются, либо один вложен в другой.

Доказательство

Пусть для определённости $\mathrm{pre}(u) < \mathrm{pre}(v)$.

Случай $\mathrm{pre}(v) > \mathrm{post}(u)$. Вызов u закрылся до обнаружения v, интервалы не пересекаются.

Случай $\mathrm{pre}(v) < \mathrm{post}(u)$. v открыта при живом вызове u, значит v — потомок u в дереве обхода. Все потомки закрываются раньше предка, поэтому $\mathrm{post}(v) < \mathrm{post}(u)$, и интервал v вложен в интервал u.

Интервалы трассировки

Пример (Интервалы прогона)

В прогоне решётки вершина 1 живёт в интервале $[1, 12]$, накрывающем всех.

Цепь вложенности начинается с $[2, 11]$ у 2, внутри лежит $[3, 10]$ у 3, внутри того $[4, 9]$ у 6, затем $[5, 8]$ у 5 и $[6, 7]$ у 4.

Ребро $2 — 5$ оказалось обратным при переборе соседей вершины 5. Сосед 2 был тогда серым, то есть лежал на текущем пути от корня.

Вложенность интервалов подтверждает то же самое. Интервал $[5, 8]$ лежит внутри $[2, 11]$, значит 5 — потомок 2, а ребро ведёт вверх к предку.

Вложенность — это карта рекурсии, она показывает, кто в чьей ветке стека находился.

На ней держатся поиск циклов, мостов и топологическая сортировка.

Глубина рекурсии

DFS живёт в стеке вызовов, и стек не бесконечен.

Пример (Предел рекурсии)

В Rust глубина рекурсии ограничена размером стека, поэтому на пути из 10^3 вершин рекурсивный обход может упасть с переполнением стека.

Явный стек доводит путь из 10^5 вершин до конца, а память остаётся $O(V)$.

BFS глубиной не страдает, его память — это ширина слоя. DFS держит в памяти путь, и его длина равна глубине.

На звезде очередь BFS хранит все листья сразу, а стек DFS — три вершины.

BFS против DFS

Задача	BFS	DFS	Почему различие
Кратчайшие пути	да	нет	очередь упорядочена по расстоянию
Циклы	недревесное ребро	ребро в серую вершину	в орграфе недревесное ребро циклом ещё не является
Топологический порядок	алгоритм Кана	обратный postorder	выход позже достижимых
Компоненты	да	да	любой обход собирает компоненту
Память	$O(V)$, это слой	$O(V)$, это путь	на звезде очередь держит все листья, на пути стек уходит на всю глубину

Различие обходов в том, что попадает в память. У BFS это граница волны, у DFS текущий путь.

Топологическая сортировка

Ориентированный граф без циклов называется **DAG** (*directed acyclic graph*).

- Зависимости сборки, где модуль a собирается раньше модуля b.
- Пререквизиты курсов, где курс a открывается раньше курса b.
- Порядок одевания, где носки надеваются раньше ботинок.

Определение 2 (Топологический порядок)

Топологическим порядком называют нумерацию вершин DAG, при которой каждое ребро $u \to v$ идёт от меньшего номера к большему.

Для DAG топологический порядок существует, для графа с циклом его нет. Цикл $a \to b \to c \to a$ не допускает нумерации, где каждая стрелка идёт вперёд.

Алгоритм Кана

Первый способ извлекает вершины без входящих стрелок. Исток не зависит ни от кого, поэтому выходит первым.

1. Посчитать входящие степени всех вершин.
2. Поставить в очередь вершины с нулевой входящей степенью — истоки.
3. Пока очередь не пуста, извлекать исток, выводить его и уменьшать степени соседей, а обнулившиеся ставить в очередь.

Пример (Прогон на зависимостях)

Граф задан рёбрами core → utils, utils → app, utils → tests. Входящие степени в порядке core, utils, app, tests равны $0, 1, 1, 1$.

Извлечение core обнуляет степень utils, а тот освобождает app и tests. Вывод core, utils, app, tests топологичен.

Топологический порядок через DFS

Второй способ — полный DFS и выписывание вершин по убыванию времени выхода.

Теорема 5 (Порядок по времени выхода)

В DAG список вершин по убыванию post топологичен — для каждого ребра $u \to v$ вершина u стоит раньше v.

Доказательство

Докажем для каждого ребра $u \to v$ неравенство $\mathrm{post}(u) > \mathrm{post}(v)$.

Случай v белая. DFS переходит из u в v, и потомок закрывается раньше предка.

Случай v серая. Ребро в предка замыкает цикл, которых в DAG не бывает.

Случай v чёрная. v закрылась ещё до входа в u, поэтому $\mathrm{post}(v) < \mathrm{pre}(u) < \mathrm{post}(u)$.

Трассировка на графе зависимостей

Пример (Обратный postorder на DAG)

Дан граф с рёбрами core → utils, utils → app, utils → tests, соседей берём по алфавиту.

Единственный исток — core, и входы идут цепочкой $\mathrm{pre}(\mathrm{core}) = 1$, $\mathrm{pre}(\mathrm{utils}) = 2$, $\mathrm{pre}(\mathrm{app}) = 3$, $\mathrm{pre}(\mathrm{tests}) = 5$.

Выходы дают $\mathrm{post}(\mathrm{app}) = 4$, $\mathrm{post}(\mathrm{tests}) = 6$, $\mathrm{post}(\mathrm{utils}) = 7$, $\mathrm{post}(\mathrm{core}) = 8$.

По убыванию выходов выходит core, utils, tests, app, и каждая стрелка смотрит вперёд.

Порядок отличается от результата Кана перестановкой app и tests — оба верны, пути между этими вершинами нет.

На графе с циклом список не будет топологическим. Провал виден по двум признакам. У DFS это ребро в серую вершину, у Кана — меньше n выведенных вершин, ведь у оставшихся степень так и не обнулилась. Сборка тогда невозможна.

Зависимости в инженерии

Пример (Сборка и пакетные менеджеры)

Сборщик `make` читает граф целей, где цель — вершина, а зависимость — стрелка.

`npm` и `cargo` разрешают граф зависимостей так же, сначала берутся зависимости, потом зависящие от них.

Цикл сборщик не пропустит. `make` сообщает о circular dependency, менеджеры пакетов отказывают в установке.

Разрешение зависимостей — это топологическая сортировка, в которой ни один шаг не выполняется раньше своих предпосылок.

Топологический порядок фиксирует лишь частичный порядок, поэтому независимые цели `make -j` запускает параллельно.

BFS в коде

```rust
use std::collections::VecDeque;
fn bfs(adj: &[Vec<usize>], start: usize) {
    let mut parent = vec![None; adj.len()];
    let mut seen = vec![false; adj.len()];
    let mut order = vec![];
    let mut q: VecDeque<_> = [start].into();
    seen[start] = true;
    while let Some(u) = q.pop_front() {
        order.push(u);
        for &v in &adj[u] {
            // пометка при входе в очередь
            if !seen[v] { seen[v] = true; parent[v] = Some(u); q.push_back(v); }
        }
    }
    println!("order {order:?}\nparent {parent:?}");
}
```

BFS в коде [2]

Сложность не зависит от формы графа, ведь каждая вершина входит в очередь один раз, а каждое ребро просматривается дважды.

DFS в коде

Рекурсивная версия повторяет таблицу трассировки, и стек вызовов виден прямо в сигнатуре.

```
fn dfs(u: usize, adj: &[Vec<usize>], seen: &mut [bool],
    parent: &mut [Option<usize>], order: &mut Vec<usize>) {
    seen[u] = true;
    order.push(u);
    for &v in &adj[u] {
        if !seen[v] {
            parent[v] = Some(u);
            dfs(v, adj, seen, parent, order);
        }
    }
}
```

DFS в коде [2]

Здесь нет времён pre и post, поэтому вложенность веток и порядок выхода не записаны. Оба времени дописываются двумя строками, на входе и после цикла соседей.

Вывод обходов в коде

Пример (Вывод на решётке)

BFS печатает порядок $1, 2, 4, 3, 5, 6$ и родителей $[-, 1, 2, 1, 2, 3]$ — запись с номером v отвечает родителю вершины v.

DFS печатает $1, 2, 3, 6, 5, 4$ — в точности порядок спуска из таблицы трассировки.

Флаг `seen` включается при постановке в очередь — тот самый инвариант трассировки.

Без него вершина встанет в очередь по разу на каждое входящее ребро.

Топология и код

1. Что означает встреча с серой вершиной в ориентированном графе?
2. Почему полный DFS для леса не портит линейность обхода?

Обходы в системном коде

- Сборщик мусора помечает достижимые объекты от корней обходом, а недостижимые освобождает (mark-and-sweep).
- В инкрементальном сборщике те же три цвета означают белый объект-кандидат, серый объект в очереди и чёрный достигнутый.
- Компилятор ищет недостижимые блоки в графе потока управления тем же обходом от входа в функцию.
- Каскадное удаление в базе данных обходит граф внешних ключей от удаляемой записи.

> «Пометить вершину» в коде означает выставить флаг `visited`, без него обход зациклится на первом же цикле.

Деревья

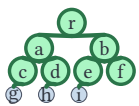
> *Родственные связи всех существ одного класса иногда изображали в виде великого дерева. Я верю, что эта метафора во многом соответствует истине.*
>
> ***— Чарльз Дарвин***

Дерево и лес

Обходы оставляли рёбра к белым вершинам, и выходил связный граф без циклов.

Определение 3

Дерево — связный граф без циклов, **лес** — граф без циклов.



Пример

Цепочка $1 - 2 - 3 - 4 - 5$ — дерево с 5 вершинами и $4 = 5 - 1$ рёбрами. Два треугольника дают 6 рёбер на 6 вершин, и два из них замыкают циклы.

Сколько рёбер у дерева?

Теорема 6 (Число рёбер)

Дерево на n вершинах имеет ровно $n - 1$ ребро.

Доказательство

Докажем индукцией по n, отделяя лист.

Лист. У конца самого длинного пути один сосед, иначе путь удлинится или замкнётся.

База. При $n = 1$ рёбер нет.

Шаг. Лист отделяется вместе с ребром, и по индукции остаётся дерево с $n - 2$ рёбрами, а с ребром листа выходит $n - 1$.

Дерево — минимальный связный каркас, и лишнего ребра в нём нет.

Пять характеризаций

Дерево узнаётся по любому из пяти признаков, и каждый удобен в своей задаче.

Теорема 7 (Эквивалентные характеризации дерева)

Для графа с n вершинами эквивалентны пять условий.

1. связный и ациклический.
2. связный и имеет $n - 1$ ребро.
3. ациклический и имеет $n - 1$ ребро.
4. между любой парой вершин — единственный простой путь.
5. связный, и каждое ребро — мост.

Пример (Проверка на пути)

Путь $1 - 2 - 3 - 4 - 5$ связен, ацикличен, и рёбер у него $4 = 5 - 1$. После добавления ребра $1 - 5$ рёбер стало 5, появился цикл, и условия от 2 до 5 нарушились. Условие 1 тоже нарушилось, ведь граф перестал быть ациклическим.

Цепочка эквивалентностей

Достаточно замкнуть круг из пяти импликаций.

Доказательство

(1) ⇒ (2). Связность дана, а рёбер ровно $n - 1$ по предыдущей теореме.

(2) ⇒ (3). Если есть цикл, вырежем лишние рёбра до дерева, и связность удержится, но у дерева $n - 1$ ребро, а рёбер стало меньше.

(3) ⇒ (4). Компонента без циклов — дерево, рёбер всего $n - c$, поэтому $n - 1$ даёт $c = 1$. Два разных простых пути дали бы цикл в симметрической разности.

(4) ⇒ (5). Пути между парами существуют, значит граф связен, а без ребра $u - v$ пропадает единственный путь между u и v.

(5) ⇒ (1). Связность дана, а ребро цикла мостом не бывает.

Деревья вокруг

Пример (Деревья вокруг)

- Файловая система, где каталоги и файлы образуют дерево с корнем `/`, а у каждого файла ровно один родительский каталог.
- DOM страницы, где теги и вложенность дают дерево, а у элемента ровно один непосредственный родитель.
- Наследование классов в языках с одиночным наследованием, где базовый класс — корень.
- Турнирная сетка плей-офф, где матч — вершина, а следующий матч победителя — его родитель.

В каждом случае у каждого объекта ровно один «родитель», а корень — начало.

Родитель всегда стоит уровнем ближе к корню, поэтому циклы исключены, а достижимость от корня даёт связность — требования дерева в чистом виде.

Остовное дерево

Каркас, который оставлял обход, получает отдельное имя.

Определение 4

Остовное дерево (*spanning tree*) графа — подграф на всех вершинах, являющийся деревом.

Теорема 8 (Существование)

Любой связный граф имеет остовное дерево.

Доказательство

Обойдём граф, оставив рёбра к белым вершинам. Каждая вершина, кроме старта, приходит по своему ребру, поэтому рёбер ровно $n - 1$ и граф связен, а по характеризации (2) это дерево.

Сколько остовных деревьев?

Деревьев на одном и том же множестве вершин много, и Кэли их пересчитал.

Теорема 9 (Формула Кэли)

У полного графа K_n на вершинах $1, \ldots, n$ ровно n^{n-2} различных остовных деревьев.

Пример

У K_4 получается $4^2 = 16$ остовных деревьев.

У K_{10} их уже 10^8, поэтому минимальный остов ищут алгоритмом, а не перебором.

Кэли вывел формулу в 1889 году.

Крускал и Прим

Рёбрам приписаны веса — ищем остов минимального суммарного веса.

- **Крускал** сортирует рёбра и добавляет легчайшее из тех, что не создают цикла.
- **Прим** растит дерево из одной вершины, добавляя легчайшее ребро наружу.

Оба алгоритма жадные, и оба оптимальны.

Жадность оптимальна не везде. Здесь работает обменный аргумент — выбранное ребро вписывается в минимальный остов заменой ребра не меньшего веса, и стоимость остова не растёт.

Проверка «не создаёт ли ребро цикла» в Крускале — это система непересекающихся множеств, где компоненты леса сливаются по мере добавления рёбер.

Остов минимального веса на числах

Пять городов соединяют семь дорог.

Шаг	Ребро	Цена	Судьба
1	$2-3$	1	в остов
2	$1-2$	2	в остов
3	$1-3$	3	цикл $1-2-3$
4	$2-4$	4	в остов
5	$3-4$	5	цикл $2-3-4$
6	$4-5$	6	в остов
7	$3-5$	7	остов уже собран

Остов $1-2$, $2-3$, $2-4$, $4-5$ стоит $1+2+4+6=13$.

Прим из вершины 1 берёт рёбра в порядке $1-2$ (цена 2), $2-3$ (цена 1), $2-4$ (цена 4), $4-5$ (цена 6). Набор рёбер совпадает с Крускалом, а порядок другой.

Проверьте себя

Вопросы

1. Почему BFS находит кратчайшие пути, а DFS — нет?
2. Что означает вложенность интервалов pre и post? Что она говорит о рекурсии?
3. Докажите, что в дереве с $n \geq 2$ есть хотя бы два листа.
4. Объясните, почему граф связен тогда и только тогда, когда в нём есть остовное дерево.
5. В каком порядке сборщик обработает цели, если зависимости образуют цикл?

Обходы и пути

Обходы дали графу порядок посещения, и на обоих взглядах выросли алгоритмы.

- Обходы идут за $O(V + E)$, где BFS работает очередью и слоями, а DFS стеком и временами pre и post.
- BFS даёт кратчайшие пути в невзвешенном графе, DFS — циклы и топологический порядок, алгоритм Кана и обратный postorder — за линейное время.
- Дерево — связный ациклический граф с $n - 1$ ребром, единственным путём между парой вершин и всеми рёбрами-мостами.

Что почитать?

Задача	Где читать
Понять BFS	m09, разделы «Обходы графа» и «Поиск в ширину»
Понять DFS	m09, раздел «Поиск в глубину»
Сортировать зависимости	m09, раздел «Топологическая сортировка»
Разобрать деревья	m09, разделы «Деревья» и «Минимальные остовные деревья»